



南開大學
Nankai University

计算机学院
并行程序设计课程报告

Pthread & OpenMP 编程: ANN

姓名：黄泽恺

学号：2411268

专业：计算机科学与技术

2026 年 5 月 25 日

目录

1 摘要	1
2 实验平台信息与源码链接	1
3 实验问题描述	1
3.1 ANNS 向量检索问题	1
3.2 评价指标	1
4 Flat-SIMD + 多线程并行	2
4.1 Pthread 优化	2
4.1.1 程序实现思路	2
4.1.2 实验结果呈现及分析	3
4.2 OpenMP 优化	4
4.2.1 程序实现思路	4
4.2.2 实验结果呈现及分析	4
5 PQ-SIMD + 多线程并行	5
5.1 Pthread 优化	5
5.1.1 程序实现思路	5
5.1.2 实验结果呈现及分析	6
5.2 OpenMP 优化	7
5.2.1 程序实现思路	7
5.2.2 实验结果呈现及分析	7
6 IVF-SIMD + 多线程并行	7
6.1 IVF-SIMD 实现	7
6.1.1 程序实现思路	7
6.1.2 实验结果呈现及分析	9
6.2 Pthread 优化	10
6.2.1 程序实现思路	10
6.2.2 实验结果呈现及分析	11
6.3 OpenMP 优化	12
6.3.1 程序实现思路	12
6.3.2 实验结果呈现及分析	13
7 IVF-PQ-SIMD + 多线程并行	14
7.1 IVF-PQ-SIMD 实现	14
7.1.1 程序实现思路	14
7.1.2 实验结果呈现及分析	15
7.2 Pthread 优化	16
7.2.1 程序实现思路	16
7.2.2 实验结果呈现及分析	17

7.3	OpenMP 优化	18
7.3.1	程序实现思路	18
7.3.2	实验结果呈现及分析	19
8	HNSW	20
8.1	程序实现思路	20
8.2	实验结果呈现及分析	22
9	实验总结与思考	23

1 摘要

本次实验围绕 ANNS 任务展开, 在 SIMD 向量化优化的基础上, 进一步使用 Pthread 与 OpenMP 对多种向量检索算法进行并行优化与性能分析。实验以 Top-k 检索为核心问题, 以平均查询延迟 latency 与检索准确率 recall 为主要评价指标, 从 Flat-SIMD、PQ-SIMD、IVF-SIMD、IVF-PQ 和 HNSW 五个角度展开, 分别进行 Pthread 和 OpenMP 多线程优化, 探索不同算法的计算密度、访存模式、任务粒度、负载均衡和同步开销对程序性能的影响。

在 Flat-SIMD 部分, 实验采用 base 向量划分并行, 多线程取得了较好的近线性加速; 在 PQ-SIMD 部分, 实验主要对 LUT 构建阶段进行簇中心集合并行优化, 同时也尝试对 ADC 查表累加阶段进行并行化, 但由于该阶段单次计算量小, 额外开销大, 可能出现负优化; 在 IVF-SIMD 部分, 实验通过倒排索引减少精排阶段扫描的候选向量数量, 并对 nlist、nprobe 进行参数调节, 在多线程优化中, 分别实现了候选簇扫描并行、动态任务划分和 query 级并行, 整体加速效果明显; 在 IVF-PQ-SIMD 部分, 实验结合 IVF 的候选筛选能力和 PQ 的近似距离计算能力降低查询开销, 在并行优化中, 实现了 cluster 并行、centroid + cluster 并行和 query 级并行, 其中 IVF-PQ 的单 query 内部并行容易受到 memory-bound 限制, 而 query 级并行仍具有稳定加速效果; 在 HNSW 部分, 实验基于 hnswlib 实现标准分层搜索和 Layer0 底层图搜索, 并进一步实现了多入口并行搜索, 适合于高 recall 需求的场景。

总体来看, 本次实验验证了不同 ANNS 方法在多线程环境下的性能特点。对于计算密集、任务均匀的阶段, 多线程能够显著降低查询延迟; 而对于访存密集或任务粒度较小的阶段, 过度并行可能导致负优化。因此, 合理选择并行粒度、线程数和算法参数, 是提升 ANNS 检索性能的关键。

2 实验平台信息与源码链接

本次实验在 OpenEuler 服务器上进行。

本次实验已上传 Github https://github.com/Redamancykai/NKU_Parallel/tree/main/Lab3

3 实验问题描述

3.1 ANNS 向量检索问题

近似最近邻搜索 (Approximate Nearest Neighbor Search, ANNS) 是高维向量检索中的核心问题。给定一个包含 N 条 d 维向量的数据库 $X = \{x_1, x_2, \dots, x_N\}$, $x_i \in \mathbb{R}^d$, 以及一个查询向量 $q \in \mathbb{R}^d$, 最近邻搜索的目标是在数据库中找到与 q 距离最近的 Top- k 个向量。

本次实验的核心任务是在已有 SIMD 优化的基础上, 进一步使用 Pthread 和 OpenMP 对 ANNS 查询过程进行多线程并行优化, 并分析不同并行策略对 latency 和 recall 的影响。

3.2 评价指标

本实验主要采用 latency 和 recall 两个指标评价检索方法的性能。

Latency 表示单个查询或一批查询的平均查询延迟, 反映算法的实际运行效率。

Recall@k 用于衡量近似检索结果与精确检索结果之间的一致程度, 定义为

$$Recall@k = \frac{|KNN(q) \cap KANN(q)|}{k},$$

其中 $KNN(q)$ 表示精确搜索得到的结果集合, $KANN(q)$ 表示使用 ANNS 得到的结果集合。

4 Flat-SIMD + 多线程并行

4.1 Pthread 优化

4.1.1 程序实现思路

在 SIMD 实验中 Flat-SIMD 的基础上, 进一步使用 Pthread 进行并行优化, 我们在此处考虑 base 向量划分并行, 即在同一个 query 下, 不再由单线程顺序扫描全部 base 向量, 而是将 base 按区间划分给不同线程共同扫描。各个线程维护自己的 top-p 堆, 在最终完成合并输出 top-k。

Listing 1: 线程计算函数

```

1 static inline void flat_worker_compute(FlatThreadParam* param) {
2     const float* base = param->base;
3     const float* query = param->query;
4     const size_t vecdim = param->vecdim;
5     const size_t local_p = param->local_p;
6
7     auto& heap = *(param->local_heap);
8
9     // 计算内积并维护局部 top-p 堆
10    for (size_t i = param->start_id; i < param->end_id; ++i) {
11        const float* base_vec = base + i * vecdim;
12        float dot = InnerProductSIMD(base_vec, query, vecdim);
13        float dis = 1.0f - dot;
14        flat_push_topk(heap, dis, static_cast<uint32_t>(i), local_p);
15    }
16 }
```

在每个线程内部我们会调用 `flat_worker_compute()` 函数使用 SIMD 指令加速向量内积计算, 每个线程只负责一部分 base 向量的扫描, 维护自己的 top-p 堆。

Algorithm 1 Flat-SIMD + Pthread 查询流程

Input: 查询向量 q , 原始数据库 X , 数据库规模 N , 向量维度 d , 局部候选数量 p , 返回数量 k

Output: Top- k 最近邻结果 R

- 1: 将数据库 X 按照 base 向量编号划分为 8 个连续区间
 - 2: 创建 7 个 pthread 子线程, 主线程作为第 0 个工作线程参与计算
 - 3: **for** 每个线程 $t = 0, 1, \dots, 7$ **并行执行 do**
 - 4: 初始化当前线程
 - 5: 调用 `flat_worker_compute(param)` 维护局部最大堆 H_t
 - 6: **end for**
 - 7: 主线程等待 7 个 pthread 子线程全部完成
 - 8: 初始化全局最大堆 H
 - 9: **for** 每个线程的局部堆 H_t **do**
 - 10: 维护全局最大堆 H
 - 11: **end for**
 - 12: 从全局堆 H 得到最终 Top- k 结果 R
 - 13: **return** R
-

多线程并行时我们采用静态分块, 这里就是将 base 向量 8 等分交给 8 个线程处理。同时由于我们只有一个 8 核的 CPU, 因此我们选择启动 7 个子线程, 主线程在创建完子线程之后也承担一部分

计算，最后将所有的 `local_heap` 合并为 `global_heap` 的 top-k。

此时需要考虑 top-p 的 p 取值对结果造成的影响，因为有可能因为 p 的取值太小 ($p \leq k$) 造成某个线程中理应进入最终 top-k 的元素没有进入局部的 top-p 导致 recall 下降，我们会通过实验进行 trade-off。

4.1.2 实验结果呈现及分析

我们针对 top-p 参数 p 对程序正确性以及性能的影响做了详细的 trade-off 分析。显然，当 $p \geq k$ 时，每个应该进入 top-k 的向量也一定会进入局部的 top-p，因此它们的 recall 也应该和串行版本的 Flat-SIMD 一致；而当 $p < k$ 时，可能因为 top-k 目标都集中在某一个线程导致结果遗漏，recall 下降，但同时由于 p 的下降，最终主线程进行合并时计算量也会下降，latency 理论上会降低。我们的 trade-off 实验结果如图 4.1 所示。

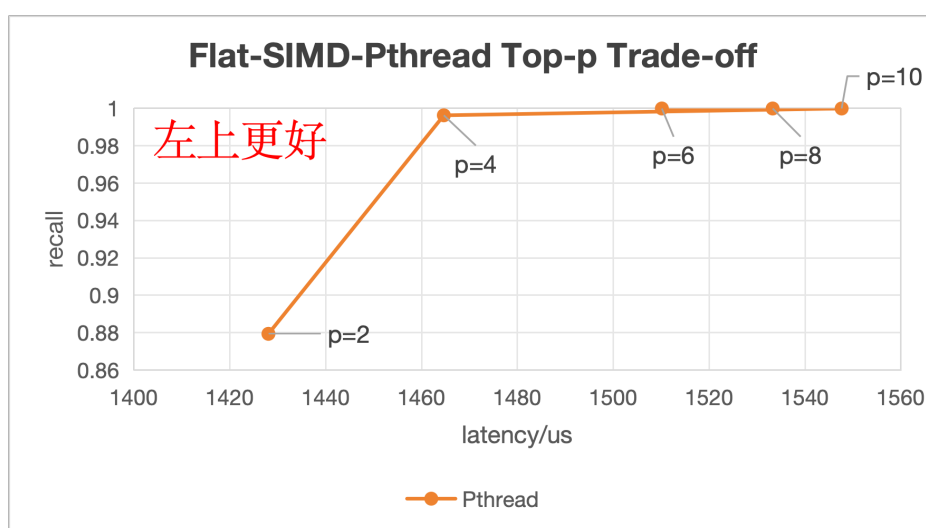


图 4.1: Flat-SIMD-Pthread 的 Latency-Recall Trade-off 曲线

如图 4.1 所示，当 $p \geq 4$ 时，recall 都近似于 1，而当 $p = 2$ 时 recall 骤减，可见过小的 p 会极大的影响检索的正确性。但同时我们也发现，盲目的降低 p 对 latency 的影响也很小，所呈现的 latency 波动很大一部分来自于服务器本身的波动，主要原因是 p 的大小只影响最终合并阶段的计算量，对于 8 线程而言， p 每下降 1 只减少了 8 个数据进入最终的合并，和之前庞大的遍历 base 向量的计算而言微乎其微。因此下面我们取 $p = 4$ 的数据进行后续的比较，实验数据如表 1 所示。

如果我们和后续的 OpenMP 对比会发现 Pthread 的加速效果明显落后一截，这是因为现在的程序对于每一个 query 都会执行一次线程的创建与销毁，这会带来大量额外开销，削弱多线程加速效果。OpenMP 则会维护一个线程池，在整个检索过程中只创建一次工作线程，在多个 query 之间复用线程，从而减少额外开销。

表 1: Flat-SIMD-Pthread 不同参数 p 的性能对比

p	latency / us	recall
10	1533.24	0.99995
8	1510.13	0.99995
6	1484.62	0.99995
4	1464.65	0.996301
2	1449.34	0.87936

4.2 OpenMP 优化

4.2.1 程序实现思路

OpenMP 优化的核心思想与 Pthread 版本一致，都是对 Flat-SIMD 的 base 向量扫描过程进行并行化。但 OpenMP 作为一个自动化工具，可以通过预编译指令自动完成线程创建、循环划分和同步。下面是利用 OpenMP 实现的核心循环：

Listing 2: OpenMP 核心循环

```

1 #pragma omp parallel
2 {
3     int tid = omp_get_thread_num();
4     auto& local_heap = local_heaps[tid];
5
6     #pragma omp for schedule(static)
7     for (size_t i = 0; i < base_number; ++i) {
8         const float* base_vec = base + i * vecdim;
9         float dot = InnerProductSIMD(base_vec, query, vecdim);
10        float dis = 1.0f - dot;
11        flat_omp_push_topk(local_heap, dis, i, local_p);
12    }
13 }
```

其中，我们通过 `#pragma omp parallel` 实现并行区域的创建以及自动创建线程，和 Pthread 相比不需要手动 `create` 和 `join`；通过 `#pragma omp for schedule(static)` 实现 `for` 循环分配给多个线程执行，用 `schedule(static)` 指定循环任务的划分方法，不需要再像 Pthread 一样手动切分 base 向量。

4.2.2 实验结果呈现及分析

OpenMP 很方便进行不同线程数的性能测试对比，并且其相比 Pthread，自动化让它具备了更好的性能，为了统一 recall 进行测试，我们在 $p = 10$ 的条件下改变线程数，因为在 $p < k$ 的情况下，修改线程数会导致 recall 不相同。实验结果如图 4.2 所示。

由图 4.2 可以明显发现随着线程数每提高一倍，程序的 latency 也近似减小一倍，可见多线程对于程序性能的提升是巨大的，其具体表现可参见表 2。

表 2: Flat-SIMD-OpenMP 不同线程数的性能对比

线程数	latency / us	recall	加速比 (对比 OpenMP 单线程)
1	5272.41	0.99995	-
2	2684.11	0.99995	1.96
4	1509.67	0.99995	3.49
8	694.035	0.99995	7.60

从实验结果可以看出，随着 OpenMP 线程数从 1 增加到 8，加速比达到 7.60，整体上已经接近线性加速，这是因为 OpenMP 并行区域本身会引入线程调度和同步开销，并且多线程同时访问内存时会竞争内存带宽和 cache 资源。但实验结果总体表明我们的 Pthread 优化与 OpenMP 优化是有效的。

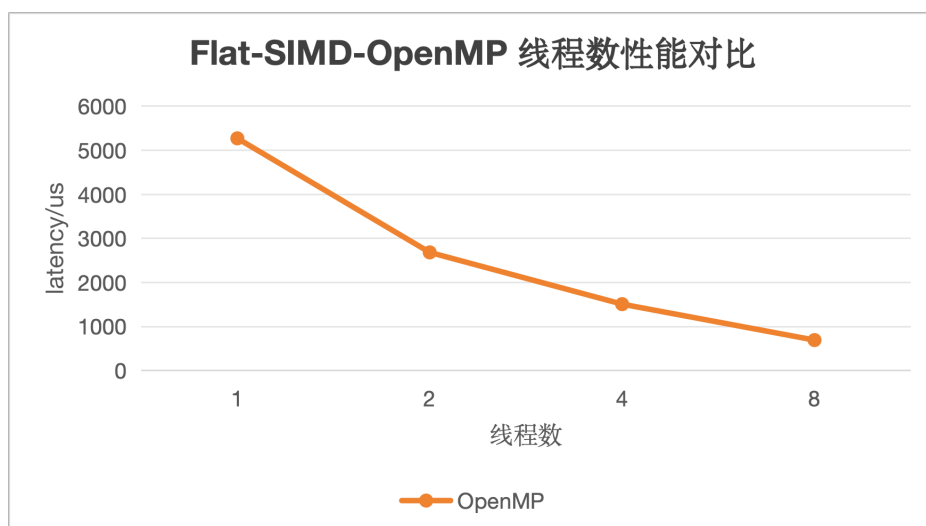


图 4.2: Flat-SIMD-OpenMP 不同线程数的性能对比

5 PQ-SIMD + 多线程并行

5.1 Pthread 优化

5.1.1 程序实现思路

在 PQ-SIMD 的基础上，本实验进一步引入 Pthread 对查询阶段进行优化。PQ 查询主要由 LUT 构建、ADC 查表粗排和原始向量精排三个阶段组成。其中 LUT 构建阶段需要计算 query 的每个子向量到对应子码本中所有聚类中心的距离，本质上仍属于 compute dense，适合进行线程级并行优化。因此程序从类中心集合并行的角度，将每个子空间中的聚类中心按照 4 个为一组划分为 centroid block 交给多线程处理。

Listing 3: 线程计算函数

```

1 void compute_lut_slice(size_t tid, size_t nthr, const float* query, float* lut) const
2 {
3     // 将 centroids 划分成 block
4     const size_t centroid_block = 4;
5     const size_t blocks_per_subspace = (Ks + centroid_block - 1) / centroid_block;
6     const size_t total_blocks = M * blocks_per_subspace;
7     // 采用静态线程划分
8     const size_t begin = total_blocks * tid / nthr;
9     const size_t end = total_blocks * (tid + 1) / nthr;
10    // compute_lut_centroid_block 内部采用 SIMD 指令加速一次计算一个 block
11    for (size_t task = begin; task < end; ++task) {
12        size_t m = task / blocks_per_subspace;
13        size_t block_id = task % blocks_per_subspace;
14        size_t c_start = block_id * centroid_block;
15        compute_lut_centroid_block(query, lut, m, c_start);
16    }
17 }

```


在线程实现上，为了避免每次查询时反复创建和销毁线程的额外开销，我们创建了一个静态线程池，每个 worker 线程在循环中监听 task_epoch 的变化，当主线程发布新的任务后，worker 线程读取所需参数并完成自己的计算。这样可以避免频繁 pthread_create 和 pthread_join 带来的额外开销。

至于查表累加阶段的 ADC 并行，由于 ADC 每个 base 的计算量很小，只有几次查表和几次加法，单个任务计算量太低，容易 memory-bound，多线程极易受到线程调度、堆维护、cache miss 等等一系列问题导致最终负优化，我们会在实验阶段进一步说明。

5.1.2 实验结果呈现及分析

首先我们针对 PQ-SIMD-Pthread 的簇中心集合并行多线程优化效果进行验证，结果如表 3 所示。

表 3: PQ-SIMD-Pthread 簇中心集合并行性能对比

线程数	latency / us	recall	加速比 (对比 Pthread 单线程)
1	1182.3	0.951904	-
2	1108.77	0.951904	1.07
4	1128.77	0.951904	1.05
8	1107.12	0.951904	1.07

从表 3 可以看出，对 PQ-SIMD 的 LUT 构建阶段进行 Pthread 簇中心集合并行后，总体表现出了稳定的加速效果，但效果并不明显。虽然 LUT 构建阶段中各个子空间、各个聚类中心之间的距离计算相互独立，适合进行线程级并行，但该阶段在整个 PQ 查询流程中的占比相对有限。一次 LUT 构建的计算规模约为 $M \times K_s \times d_{sub}$ ，而后续仍需进行全体 base 编码的 ADC 查表扫描、top-p 候选维护以及 Flat-SIMD 精排。此外，Pthread 任务发布、线程同步和等待也会带来额外开销，进一步限制了加速比。

下面我们来探索针对查表累加阶段进行多线程优化究竟会不会带来负优化，同时根据实验结果进行原因分析。实验结果见表 4。

表 4: PQ-SIMD-Pthread 查表累加优化探索

线程数	latency / us	recall	加速比 (对比 Pthread 单线程)
1	1137.47	0.951904	-
2	1639.31	0.951904	0.69
4	2069.53	0.951904	0.55
8	2652.49	0.951904	0.43

从表 4 可以看出，对 ADC 查表累加阶段进行多线程并行后，性能反而明显下降，这说明 ADC 阶段并不适合直接采用 Pthread 并行。ADC 对每个 base 向量只需要进行 M 次查表和累加，单个任务计算量很小，计算密度较低，整体更偏向 memory-bound。同时，多线程版本需要每个线程维护局部 top-p 堆，并在最后将多个局部堆合并为全局 top-p 结果，引入了额外的堆维护和 merge 开销。随着线程数增加，cache 竞争、同步等待和局部堆合并开销进一步增大，导致并行版本出现负优化。

5.2 OpenMP 优化

5.2.1 程序实现思路

OpenMP 优化的核心思想与 Pthread 版本一致，这里就不再探索负优化的查表累加，重点利用 OpenMP 实现 LUT 构建的多线程优化。下面是利用 OpenMP 实现的核心循环：

Listing 4: OpenMP 核心循环

```

1 #pragma omp parallel for num_threads(actual_threads) schedule(static)
2 for (long long task = 0; task < static_cast<long long>(total_blocks); ++task) {
3     size_t m = static_cast<size_t>(task) / blocks_per_subspace;
4     size_t block_id = static_cast<size_t>(task) % blocks_per_subspace;
5     size_t c_start = block_id * centroid_block;
6
7     compute_lut_centroid_block(query, lut.data(), m, c_start);
8 }

```

其中我们采用 `schedule(static)` 静态划分线程的方法，因为根据上面 Pthread 的实验结果，这里采用 `dynamic` 可能造成不必要的额外开销，同时此处的 `compute_lut_centroid_block` 和之前一样，都采用了 SIMD 指令优化。

5.2.2 实验结果呈现及分析

我们通过实验来验证不同线程数下 OpenMP 的优化效果，实验结果如表 5 所示。

表 5: PQ-SIMD-OpenMP 簇中心集合并行性能对比

线程数	latency / us	recall	加速比 (对比 OpenMP 单线程)
1	1154.57	0.951904	-
2	1184.22	0.951904	0.97
4	1136.89	0.951904	1.02
8	1194.25	0.951904	0.97

从表 5 可以看出，PQ-SIMD-OpenMP 簇中心集合并行和 Pthread 的结果基本相差无几，可以归结于服务器波动的原因。其主要是由于这次我们的 Pthread 手动写了一个线程池，很好了弥补了一个性能上的缺点，因此 OpenMP 没有体现出明显的优势，但是其实现逻辑简单。

此外，OpenMP 的 `parallel for` 会引入线程调度、任务划分和隐式同步开销。由于本实验中每个 centroid block 的任务粒度较小，单次并行循环的计算量不足以完全摊薄这些额外开销，因此当线程数过多时反而体现不出多线程加速的优势。

6 IVF-SIMD + 多线程并行

6.1 IVF-SIMD 实现

6.1.1 程序实现思路

IVF(Inverted File) 的核心思想是先将 base 向量聚类，划分到若干个簇中，每个簇由一个聚类中心表示。查询时不再扫描全部 base 向量，而是先计算 query 与所有聚类中心的距离，选出距离 query

最近的若干个簇，即 $nprobe$ 个候选簇，然后只在这些候选簇对应的倒排链表中进行精确搜索。这样可以显著减少参与精排的向量数量，从而降低查询延迟。

对于 IVF 索引构建阶段，主要分为三步：(1) 初始化聚类中心 (2) KMeans 迭代聚类 (3) 建立倒排链表。对于前两步基本和 PQ Index 的建立很相似，其中在计算类中心距离时采用了 SIMD 指令进行加速，加快了索引构建的速度；对于最后一步倒排链表，则是在最终质心确定后，将原始 base 向量插入对应的簇内。

Listing 5: 建立倒排链表

```

1 for (size_t i = 0; i < base_number; ++i) {
2     const float* x = base + i * vecdim;
3     uint32_t cid = ivf_nearest_centroid(index, x);
4     index.inverted_lists[cid].push_back(static_cast<uint32_t>(i));
5 }

```

对于 IVF 查询阶段，主要分为两部分，即粗排阶段和精排阶段。粗排阶段即选出和 query 向量最近的 $nprobe$ 个簇，计算量近似为 $nlist$ 次距离计算 (常为 256 或 512 次)；而精排阶段则需要对选出的 $nprobe$ 个簇中的每个 base 向量都进行内积距离计算，对于 $N = 100000$ ，一般需要 $\frac{N}{nlist} \times nprobe \approx 3000$ ，很明显精排阶段是查询时 latency 的主要来源，也是我们需要重点优化的部分。

Listing 6: IVF-SIMD 精排算法

```

1 static inline std::priority_queue<std::pair<float, uint32_t>>
2 ivf_search_simd(const IVFIndex& index, const float* query, size_t k, size_t nprobe) {
3     ...
4     std::vector<uint32_t> probe_lists;
5     ivf_select_probe_lists(index, query, nprobe, probe_lists);
6     std::priority_queue<std::pair<float, uint32_t>> result;
7
8     // 遍历 nprobe 个倒排链表
9     for (uint32_t cid : probe_lists) {
10         const std::vector<uint32_t>& list = index.inverted_lists[cid];
11         // 计算内积距离维护 top-k 堆
12         for (uint32_t id : list) {
13             const float* x = index.base + static_cast<size_t>(id) * index.vecdim;
14             float dis = 1.0f - InnerProductSIMD(query, x, index.vecdim);
15             push_topk(result, dis, id, k);
16         }
17     }
18     return result;
19 }

```

其中，probe_lists 保存粗排阶段选出的候选簇编号。对于每个候选簇，程序访问其对应的倒排链表，并遍历其中所有 base 向量编号。对于每个候选向量，仍然使用 SIMD 内积函数计算其与 query 的距离，然后通过 push_topk() 维护最终的 Top-k 结果。可见 IVF-SIMD 的优化体现在两方面：

- 通过 IVF 索引减小精排阶段的计算量
- 通过 SIMD 优化的内积计算函数加快单次距离计算速度

6.1.2 实验结果呈现及分析

从前面 IVF 的原理介绍我们不难发现, $nlist$ 和 $nprobe$ 是影响 IVF 性能最重要的两个参数。 $nlist$ 表示聚类中心数量, 较大的 $nlist$ 会使每个簇中的向量数量减少, 从而降低单个簇的扫描开销, 对于相同的 $nprobe$, 精排阶段计算开销减小, 但同时也会增加构建索引和粗排阶段的计算开销。 $nprobe$ 表示查询时访问的候选簇数量, 较小的 $nprobe$ 可以减少扫描向量数量, 从而降低 latency, 但可能由于遗漏导致 recall 下降。

因此, 对于合适的 $nlist$ 和 $nprobe$ 值进行探索, 可以对程序的 latency 和 recall 进行 trade-off。

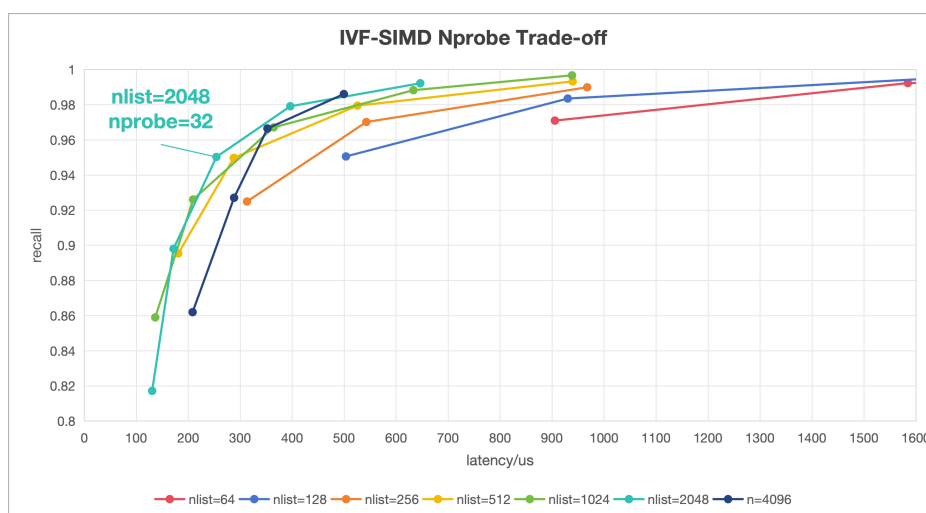


图 6.3: IVF-SIMD 的 Latency-Recall Trade-off 曲线

由图 6.3 可见, 我们大范围的探索了不同 $nlist$ 和 $nprobe$ 的组合, 希望找到一组合适的参数使得 recall 高的同时 (>0.95) latency 较低。绿色折线 ($nlist=1024$) 和青色折线 ($nlist=2048$) 都比较靠近坐标系的左上方, 证明 $nlist$ 在这两个取值下总体表现更加平衡且出色; 综合每个散点来看, 我们最后认为图中特殊标注的 $nlist = 2048$, $nprobe = 32$ 的这组参数表现最佳

这两条折线更详细的实验数据如表 6 和表 7 所示。

表 6: IVF-SIMD 不同参数 $nprobe$ 的性能对比 ($nlist=1024$)

$nprobe$	8	16	32	64	128
latency / us	136.412	209.275	364.35	633.024	938.224
recall	0.859003	0.926103	0.967152	0.988301	0.996751

表 7: IVF-SIMD 不同参数 $nprobe$ 的性能对比 ($nlist=2048$)

$nprobe$	8	16	32	64	128
latency / us	130.424	171.581	254.077	396.22	646.267
recall	0.817202	0.898103	0.950302	0.979152	0.992251

6.2 Pthread 优化

6.2.1 程序实现思路

从对 IVF 计算量的分析我们已经发现，精排阶段的计算量占据了 IVF-SIMD 查询阶段的主导地位，明显更加适合多线程优化，于是我们先打算对“簇划分并行”进行 Pthread 优化。

我们发现串行版本的 IVF 精排阶段逐个计算候选簇，但其实不同簇之间是相互独立的，读取计算互不影响，因此可以将 nprobe 个候选簇划分给多个线程，每个线程维护自己的局部 Top-k 堆；所有线程结束后，再将局部 Top-k 合并成最终 Top-k。

但是我们发现，由于不同倒排链表的长度通常并不均匀，因此如果某个线程划分到了比较大的簇可能会影响程序的性能表现，因此这里不再采用静态簇划分的方式，而是采用动态簇划分，即维护一个任务队列，每个线程处理完当前簇后，线程继续领取下一个任务，直到所有候选簇都被处理完。动态任务划分可以很好的改善负载均衡，提高多线程的执行速度。

Listing 7: 动态簇划分多线程

```

1 static void* ivf_dynamic_thread_func(void* arg) {
2     ...
3     while (true) {
4         int task_id;
5         // 领取任务，需要加锁处理
6         pthread_mutex_lock(&param->task_mutex);
7         task_id = *(param->next_task);
8         (*(param->next_task))++;
9         pthread_mutex_unlock(&param->task_mutex);
10        // 没有任务了退出
11        if (task_id >= static_cast<int>(probe_lists.size())) {
12            break;
13        }
14        // 从索引表中获取目标聚类的向量 id 和数据
15        uint32_t cid = probe_lists[static_cast<size_t>(task_id)];
16        const std::vector<uint32_t>& ids = index.inverted_lists[cid];
17        const std::vector<float>& vecs = index.inverted_vectors[cid];
18        assert(vecs.size() == ids.size() * index.vecdim);
19        const float* vec_data = vecs.data();
20        param->scanned_lists++;
21        param->scanned_points += ids.size();
22        // 线程处理当前的 list，维护 local top-k
23        for (size_t j = 0; j < ids.size(); ++j) {
24            const float* x = vec_data + j * index.vecdim;
25            float dis = 1.0f - InnerProductSIMD(query, x, index.vecdim);
26            push_topk(local_heap, dis, ids[j], param->local_k);
27        }
28    }
29    return nullptr;
30 }

```

动态簇划分的核心循环如上，这个循环可以看成两部分，第一部分是领取任务，这里由于多个线程同时访问同一个 task，所以需要 pthread_mutex_lock 加锁确保线程安全。第二部分则是计算任务，

每个线程处理自己领取到的任务，维护自己的 local_k 最大堆。

与此同时，我们还尝试探索了精排阶段的 query 并行，现在的 test 方式是一次对一个 query 进行 IVF 查询，但是每个 query 之间的查询明显是独立的，可以调用多线程处理，即一次处理 num_threads 个 query，直觉上这样可以显著加快查询速度。

6.2.2 实验结果呈现及分析

下面我们主要通过实验来验证我们对簇并行优化的效果，主要从两方面入手探索，分别是验证我们的动态 schedule 策略比静态效果更好，以及探索线程数对于程序性能的影响。下面我们在 nlist = 2048, local_k = 10 的统一条件下进行簇并行性能分析，结果如图 6.4 所示。

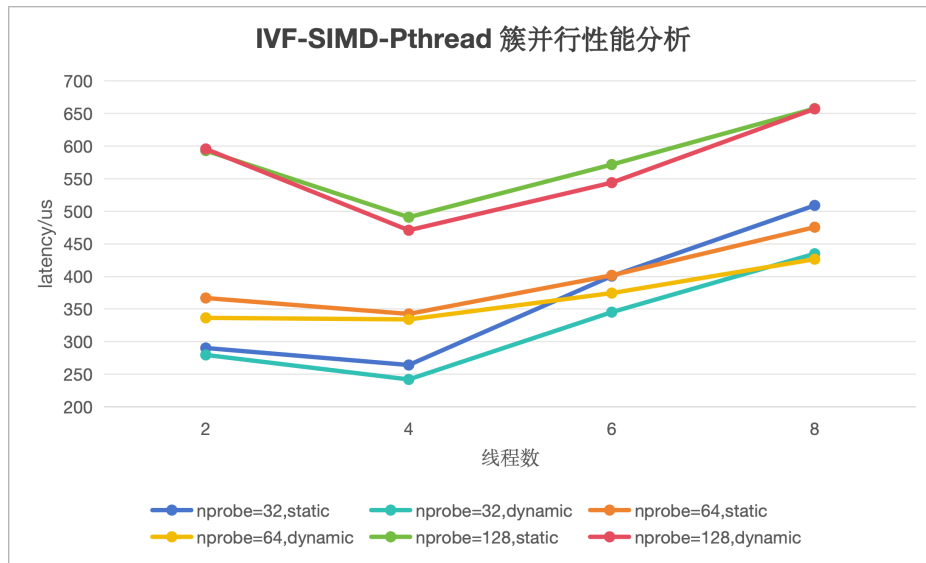


图 6.4: IVF-SIMD-Pthread schedule 和 threads 对性能的影响

实验结果表明，在不同线程数下，dynamic 整体优于 static(青色折线在蓝色折线下方，黄色折线也在橙色折线下方，红色折线在绿色折线下方)。这是因为 IVF 检索中不同倒排簇的大小可能存在较大差异，dynamic 调度能够在线程完成当前任务后继续分配新的任务，从而改善负载均衡。但同时注意到，dynamic 也会带来额外的任务分配开销，可能在任务分配均衡的情况下造成负优化或几乎没有优化，如图 6.4 中 nprobe = 128 一组中 threads = 2 和 8 的情况。

借助图 6.4，我们还可以分析线程数和程序性能的关系。显然，无论是哪种 schedule 都不意味着线程数越多性能越好。从图中可以看到，dynamic 在 4 线程时取得最低 latency，继续增加到 6 线程和 8 线程后，latency 反而上升。这主要是因为继续增加线程后，任务划分、线程调度、同步等待和局部 top-k 合并等开销逐渐增大，同时，IVF 检索过程中需要频繁访问 base 向量和倒排表数据，线程数过多会加剧 cache 竞争和内存带宽压力，导致访存开销上升。因此，额外线程带来的计算加速不足以抵消系统开销，反而会导致性能下降。

同时我也考虑到多线程加速效果和簇中心并行的任务数量，也就是 nprobe 的大小息息相关，对于线程数较多的多线程，即便是 nprobe = 128 的情况，1 个线程也只承担 16 个任务，对于我们常用的 nprobe 每个线程只承担个位数的任务，很明显它节约的时间难以抵消使用多线程的额外开销。表 8 展现了表现最优的多线程带来的性能提升，很明显随着 nprobe 增大，多线程的加速效果也越明显，这是因为精排阶段计算任务增加，更偏向 compute dense，契合多线程加速的部分，额外开销被摊薄。

表 8: IVF-SIMD-Pthread 加速效果

nprobe	threads	latency / us	recall	对照组 latency/us	加速比
32	4	241.953	0.950302	254.077	1.05
64	4	334.112	0.979152	396.22	1.19
128	4	470.916	0.992251	646.267	1.37

下面通过实验验证了 query 级并行的优化效果。我们验证了在 $nlist = 2048$, $nprobe = 32$ 的情况下不同线程数和程序性能之间的关系，结果如表 9 所示。

表 9: IVF-SIMD-Pthread query 级并行性能

threads	latency / us	recall	加速比 (对比单线程)
2	141.707	0.950302	1.79
4	74.813	0.950302	3.40
6	57.0885	0.950302	4.45
8	37.1995	0.950302	6.83

我们发现 query 的加速效果非常出色，原因在于它把不同 query 之间完全独立的搜索任务分配给不同线程，避免了单个 query 内部并行时的很多同步、负载不均和合并开销，并且也基本不受限于 memory-bound 的阻碍，加速效果明显。

6.3 OpenMP 优化

6.3.1 程序实现思路

OpenMP 优化的核心思想与 Pthread 版本一致，这里实现了 OpenMP 版本的簇划分并行和 query 级并行，下面是利用 OpenMP 实现的核心循环：

Listing 8: OpenMP 簇划分并行核心循环

```

1  #pragma omp parallel num_threads(thread_num)
2  {
3      int tid = omp_get_thread_num();
4      auto& local_heap = local_heaps[static_cast<size_t>(tid)];
5
6      #pragma omp for schedule(dynamic, 1)
7      for (int pi = 0; pi < static_cast<int>(probe_lists.size()); ++pi) {
8          uint32_t cid = probe_lists[static_cast<size_t>(pi)];
9          const std::vector<uint32_t>& ids = index.inverted_lists[cid];
10         const std::vector<float>& vecs = index.inverted_vectors[cid];
11
12         assert(vecs.size() == ids.size() * index.vecdim);
13         const float* vec_data = vecs.data();
14         scanned_points[static_cast<size_t>(tid)] += ids.size();
15
16         for (size_t j = 0; j < ids.size(); ++j) {
17             const float* x = vec_data + j * index.vecdim;
18             float dis = 1.0f - InnerProductSIMD(query, x, index.vecdim);

```



```

19         push_topk(local_heap, dis, ids[j], local_k);
20     }
21 }
22 }

```

这里使用 `#pragma omp for schedule(dynamic, 1)` 把 `for` 循环中的迭代任务动态分配给线程，对应 Pthread 里的动态任务划分。而 query 并行则使用了简单的 `static`，原因是每一个 query 的计算量基本相同，不需要额外的考虑负载均衡问题，使用 `static` 反而减小额外开销，性能更好。

Listing 9: OpenMP query 并行核心循环

```

1 #pragma omp parallel for num_threads(thread_num) schedule(static)
2 for (int qi = 0; qi < static_cast<int>(query_number); ++qi) {
3     const float* query = queries + static_cast<size_t>(qi) * vecdim;
4     cache_results[static_cast<size_t>(qi)] = ivf_search_simd(index, query, k, nprobe);
5 }

```

6.3.2 实验结果呈现及分析

下面主要通过实验来验证使用 OpenMP 对簇并行优化和 query 级并行的效果，同时还展示了 OpenMP 和 Pthread 之间的对比，探索了线程数对于程序性能的影响。下面我们在 `nlist = 2048, local_k = 10` 的统一条件下进行簇并行性能分析，结果如图 6.5 所示。

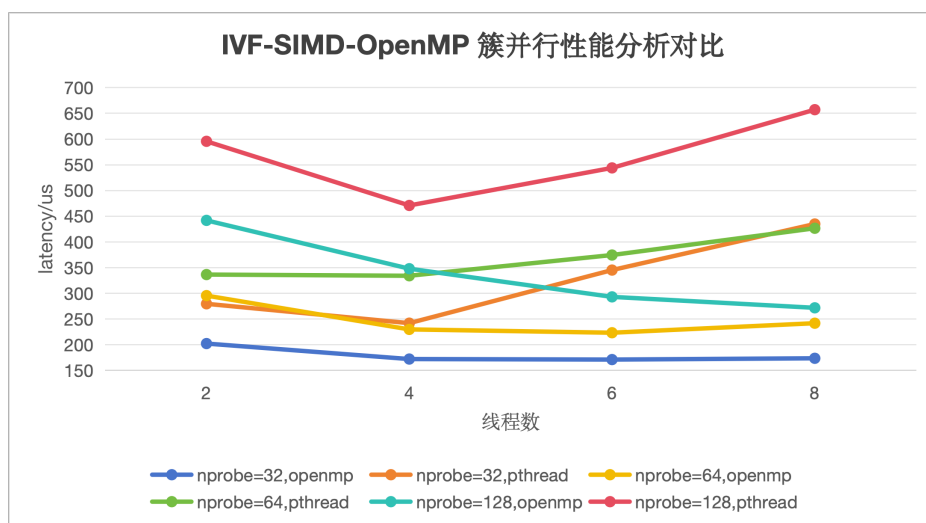


图 6.5: IVF-SIMD-OpenMP 性能分析及对比

从实验结果可以看出，在簇并行优化中，OpenMP 版本在不同 `nprobe` 和线程数下均明显优于 Pthread 版本。这主要是因为 OpenMP 运行时具有较成熟的线程池和循环调度机制，配合 `schedule(dynamic, 1)` 能够较好地缓解 IVF 倒排表长度不均衡带来的负载不均问题。而 Pthread 版本需要手动进行线程管理、任务分配和结果合并，线程同步与调度开销更明显，因此在部分线程数下甚至出现 latency 上升的现象。

同时我们发现，随着 `nprobe` 增大，OpenMP 多线程的效果也越明显，尤其是 `nprobe = 128` (青色折线)，较大的 `nprobe` 提供了更多可并行任务，使加速效果更加明显。

表 10: IVF-SIMD-OpenMP query 级并行性能

threads	latency / us	recall	加速比 (对比单线程)
2	151.302	0.950302	1.68
4	86.3085	0.950302	2.94
6	57.0895	0.950302	4.45
8	48.2725	0.950302	5.26

query 级并行实验结果如表 10 所示, OpenMP query 并行同样具有更显著的优化效果, 其原因和 Pthread 类似, 在此不再赘述。

7 IVF-PQ-SIMD + 多线程并行

7.1 IVF-PQ-SIMD 实现

7.1.1 程序实现思路

IVF-PQ 的核心目标是: 我们无需扫描所有的 base 向量, 而是先用 IVF 缩小候选范围, 再用 PQ 近似距离快速筛选, 最后只对少量候选做原始向量精排。相比 IVF 而言, 二者在粗排阶段选簇的计算量相同, 核心差距在于, 对于粗排选出来的 $nprobe$ 个簇, 如果里面一共有 S 个向量, IVF 需要完整访问这 S 个向量进行精排, 计算量为 $O(S \times D)$; 而 IVF-PQ 则是又要经过构建 PQ 和 LUT 的过程, 核心计算在于 ADC 查表累加 $O(S \times M)$ 以及最后精排阶段 $O(top_p \times D)$ 。考虑到 PQ 带来的大量额外开销, 只有在 S 足够大的情况下才能明显体现出 IVF-PQ 的优化优势。

Algorithm 2 IVF-PQ-SIMD 查询流程

Input: 查询向量 q , IVF-PQ 索引 $\mathcal{I}$, 粗聚类中心数量 $nlist$, 探测簇数量 $nprobe$, PQ 子空间数量 M , 码本大小 K_s , 粗排候选数量 p , 返回数量 k

Output: Top- k 最近邻结果 R

```

1: 初始化大小为  $nprobe$  的最大堆  $H_c$ 
2: for 聚类中心  $c = 0, 1, \dots, nlist - 1$  do
3:   计算  $(dist(q, \mu_c), c)$  并维护最大堆  $H_c$ 
4: end for
5: 初始化 PQ 查找表  $LUT$ 
6: for 每个 PQ 子空间  $m = 0, 1, \dots, M - 1$  do
7:   for 子码本中心  $s = 0, 1, \dots, K_s - 1$  do
8:      $LUT[m][s] \leftarrow dist(q_m, C_{m,s})$ 
9:   end for
10: end for
11: for 每个待扫描倒排表  $cid \in \mathcal{P}$  do
12:   for 倒排表  $cid$  中的每个向量编号  $id$  do
13:     for 每个 PQ 子空间  $m = 0, 1, \dots, M - 1$  do
14:        $approx\_dist \leftarrow approx\_dist + LUT[m][code(id, m)]$ 
15:     end for
16:     维护最大堆  $H_p$ 
17:   end for
18: end for
19: for  $H_p$  中的每个候选向量编号  $id$  do
20:   计算精确距离  $dist(q, x_{id})$  并维护最大堆  $H$ 
21: end for
22: return  $R$ 

```

这里采用的索引构建方式是先对所有 base data 进行 PQ，再构建 IVF 索引，当然我们也可以尝试先构建 IVF 索引，再在每个簇中分别进行 PQ。这两种索引构建的方法对程序的性能影响在实验部分会进行具体分析。

7.1.2 实验结果呈现及分析

下面首先来通过实验比较一下 IVF 和 IVF-PQ 的性能，我们在 $nlist = 1024$, $M = 16$, $Ks = 256$ 的参数下构建索引（无特殊说明后续相同）， $top-p = 200$ ，结果如图 7.6 所示。

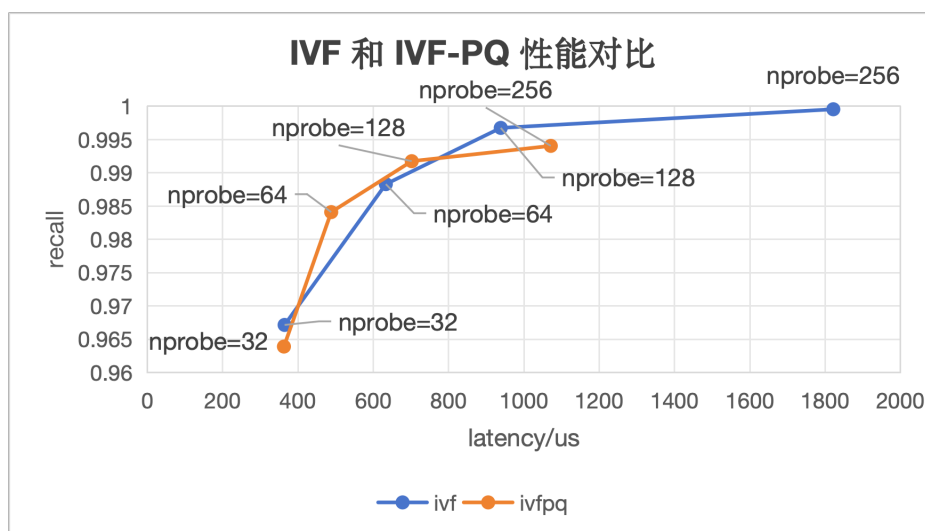


图 7.6: IVF 和 IVF-PQ 性能对比

从图 7.6 可见，二者的 recall 都随 nprobe 增大而提高，但 IVF-PQ 的性能优势主要出现在 nprobe 较大、扫描候选数量较多的情况下。当 $nprobe=32$ 时，IVF 和 IVF-PQ 的 latency 分别为 364.25 us 和 362.507 us，几乎没有差距，随着 nprobe 增大，IVF 需要对更多原始向量做完整精确距离计算，latency 增长更快；而 IVF-PQ 只对大量候选做低成本 PQ 近似距离计算，再将 top-p 候选送入精排，因此在 $nprobe=64, 128, 256$ 时分别取得约 1.30、1.34 和 1.70 倍的加速。由此可见，IVF-PQ 更适合 nprobe 较大、候选集较大的情况。

除了通过调节 nprobe 觉得进入候选集的向量数量，选择合适的 top-p 也可以一定程度上减小最终 rerank 阶段的计算开销，trade-off 曲线如图 7.7 所示。

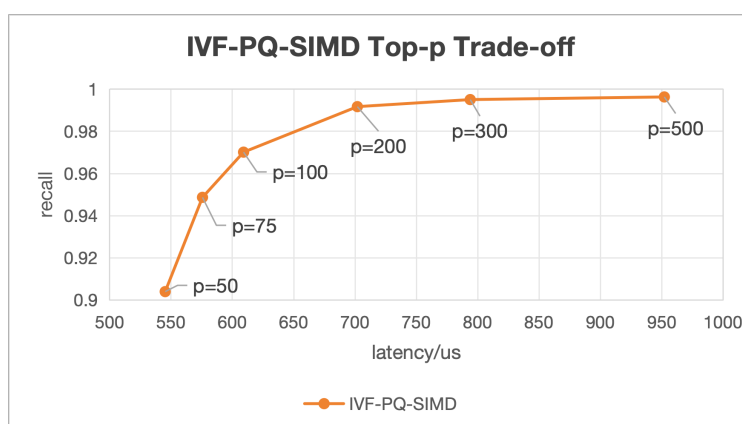


图 7.7: IVF-PQ-SIMD 的 Latency-Recall Trade-off 曲线

明显可见, p 从 50 上涨到 200 的过程中, recall 明显上升, 且 latency 的增长也相对可控, 当 p 增大到 200 时, recall 提升到 0.991751, 此时已经能够覆盖大部分高质量候选。但继续增大到 300 和 500 时, recall 的增长已经极度有限, 而 latency 已经过快增长。综合来看, $p=200$ 在本实验中是较好的折中点, 能够在约 0.99 recall 的前提下保持较低 latency。

最后我们来探讨一下不同索引构建方式对 IVF-PQ 算法的影响, 具体数据如表 11 所示。

表 11: IVF-PQ-SIMD 索引构建方式差异

方法	nprobe	latency / us	recall
先 PQ 后 IVF	32	362.507	0.963902
先 IVF 后 PQ	32	1485.87	0.967152
先 PQ 后 IVF	64	488.043	0.984102
先 IVF 后 PQ	64	2749.26	0.988301
先 PQ 后 IVF	128	701.857	0.991751
先 IVF 后 PQ	128	5224.84	0.996751

“先 IVF 后 PQ”的 recall 整体高于“先 PQ 后 IVF”, 但其性能随 nprobe 增大急剧下降。准确率方面, “先 PQ 后 IVF”通过全局 PQ 量化 base 向量, 量化误差较大, 因此可能造成 recall 损失; 而“先 IVF 后 PQ”先将向量划分到不同倒排簇, 再在局部簇内进行 PQ 表示, 簇内数据分布更加集中, 量化误差更小; 性能方面, “先 PQ 后 IVF”只需要维护一套 PQ, 而“先 IVF 后 PQ”需要处理多个局部 PQ, nprobe 增大后会带来更多局部 LUT 构建、查表访问和随机访存开销, 造成性能急剧下滑。因此在本实验中我们选择“先 PQ 后 IVF”的方式构建索引。

7.2 Pthread 优化

7.2.1 程序实现思路

考虑到 IVF-PQ 的实现流程, 我们重点从三个方面进行并行优化: 簇划分并行、聚类中心划分并行、query 级并行。

Listing 10: cluster 并行

```

1 static void* ivfpq_dynamic_thread_func(void* arg) {
2     ...
3     while (true) {
4         ... // 和 ivf 簇划分并行多线程一致
5         for (size_t pos = 0; pos < ids.size(); ++pos) {
6             // 取出目标向量的 PQ code
7             const uint8_t* code = codes_in_list.data() + pos * index.M;
8             // 查表累加计算 ADC 距离
9             float dis = ivfpq_adc_distance_code(index, code, lut);
10            ivfpq_push_topk(param->local_heap, dis, ids[pos], param->local_p);
11        }
12    }
13    return nullptr;
14 }

```

动态簇划分并行主要针对 IVF-PQ 查询中的倒排表扫描阶段进行优化。在完成 IVF centroid 粗排和 PQ LUT 构建后, 该方法将选中的 nprobe 个倒排表作为动态任务队列, 多个 Pthread 线程在各自

负责的簇内对 PQ code 执行 ADC 查表累加，得到局部 Top-p 候选集合。所有线程结束后，主线程合并各线程的局部 Top-p，并使用 Flat-SIMD 对候选向量进行精排，该方法的优势在于能够缓解不同倒排表长度不均衡带来的负载不均问题。

Listing 11: centroid 并行

```

1 static void* ivfpq_centroid_pthread_func(void* arg) {
2     ... ..
3     // 把 nlist 个聚类中心按线程编号平均切分。第 tid 个线程只处理 [start, end) 这段
        centroid
4     const size_t start = nlist * static_cast<size_t>(tid)
5                          / static_cast<size_t>(thread_num);
6     const size_t end = nlist * static_cast<size_t>(tid + 1)
7                       / static_cast<size_t>(thread_num);
8
9     for (size_t c = start; c < end; ++c) {
10        // 取出第 c 个 centroid，计算 query 与该 centroid 的内积距离
11        const float* centroid = index.centroids.data() + c * vecdim;
12        float dis = 1.0f - InnerProductSIMD(query, centroid, vecdim);
13        ivfpq_pthread_push_topk(param->local_heap, dis, static_cast<uint32_t>(c),
14                                nprobe);
15    }
16    return nullptr;
17 }
```

此外我们将 centroid 和 cluster 并行结合起来，进一步扩大了并行范围。在 centroid 阶段，所有聚类中心按照编号划分给不同线程，每个线程计算 query 到本线程负责 centroid 的距离并维护局部 Top-nprobe，随后主线程合并得到全局待扫描簇集合。在 cluster 阶段，则和之前一致。但由于 centroid 阶段本身计算量相对较小，当 nlist 不大时，其加速收益可能被线程创建、同步和局部结果合并开销抵消。因此该方法更适合 nlist 较大的情景。

最后我们也实现了 query 级并行，实现方式类似于 IVF。

7.2.2 实验结果呈现及分析

首先我们通过实验来验证 cluster 并行以及 centroid + cluster 并行的多线程优化效果，实验结果如图 7.8 所示。

从实验结果可以看出，IVF-PQ-SIMD-Pthread 的单 query 内部并行并不是线程数越多性能越好。对于 cluster 并行和 centroid+cluster 并行，2 线程和 4 线程时 latency 有一定下降，其中 centroid+cluster 在 4 线程时达到最低 latency，为 601.440 us。但当线程数增加到 6 和 8 后，两种方法都出现明显负优化，尤其 centroid+cluster 在 8 线程下 latency 上升到 1303.300 us。这说明在 IVF-PQ 中，PQ 编码已经显著降低了单次距离计算量，ADC 阶段主要表现为 PQ code 读取和 LUT 查表累加，程序瓶颈从 compute-bound 逐渐转向 memory-bound。此时继续增加线程会加重 cache 竞争和线程同步开销，导致多线程收益下降甚至变慢。

下面通过实验验证了 query 级并行的优化效果，探索不同线程数和程序性能之间的关系，实验结果如表 13 所示。

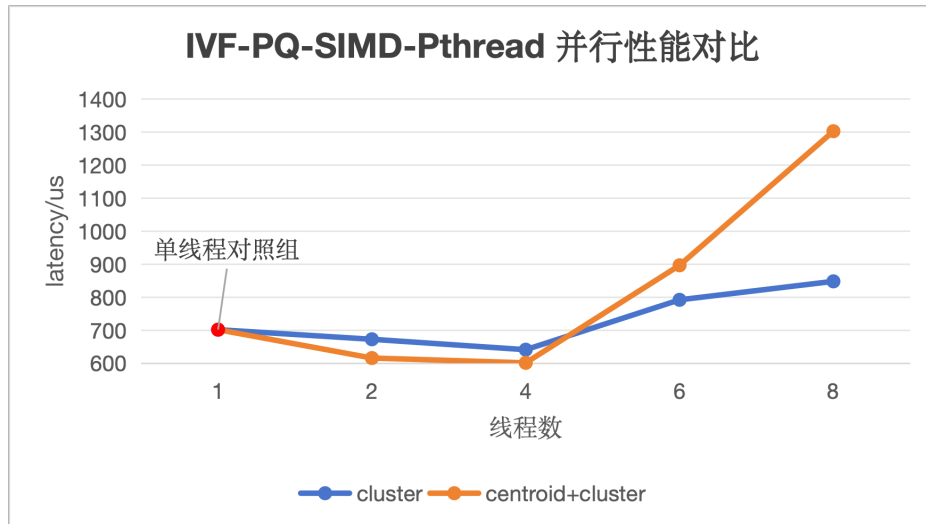


图 7.8: IVF-PQ-SIMD-Pthread 的多线程并行性能对比 (recall=0.991751)

表 12: IVF-PQ-SIMD-Pthread query 级并行性能

threads	latency / us	recall	加速比 (对比单线程)
2	366.905	0.991751	1.91
4	181.059	0.991751	3.88
6	124.455	0.991751	5.64
8	91.6235	0.991751	7.66

和之前所做的多线程类似, query 级并行表现更加稳定, 随着线程数增加, latency 也不断减小, 加速比从 1.91 提升到 7.66。query 级并行将不同 query 分配给不同线程独立执行, 任务粒度更大、同步开销更小, 因此更适合 IVF-PQ 场景。

7.3 OpenMP 优化

7.3.1 程序实现思路

OpenMP 优化的核心思想与 Pthread 版本基本一致, 同样实现了 cluster 并行、centroid + cluster 并行和 query 级并行。但是这里针对多线程的负载均衡问题、虚假共享问题、线程嵌套等等做了进一步分析处理。下面是利用 OpenMp 实现的核心循环:

Listing 12: OpenMP 核心循环

```

1 #pragma omp parallel num_threads(thread_num) default(none) \
2   shared(index, query, nprobe, top_p, thread_num, centroid_locals, adc_locals,
3     probe_lists, lut, adc_chunk_size)
4 {
5   // 并行 centroid 粗排
6   #pragma omp for schedule(static)
7   for (int ci = 0; ci < index.nlist; ++ci) {
8     ... ..
9   }
10  // single 合并 centroid local top-nprobe, 并构建 LUT

```

```

10  #pragma omp single
11  {
12      ... ..
13      ivfpq_build_lut(index, query, lut);
14  }
15  // 并行 ADC 扫描
16  #pragma omp for schedule(dynamic, adc_chunk_size)
17  for (int pi = 0; pi < probe_lists.size(); ++pi) {
18      ... ..
19  }
20  }

```

首先我们对于两部分的并行只使用了一次 `#pragma omp parallel`，如果分成两个 parallel region，每个 query 至少会产生两次 fork-join，而单 query 的任务粒度本来就不大，减少 fork-join 是提升稳定性的关键；其次对于 centroid 粗排并行我们使用 `schedule(static)`，因为每个 centroid 的计算量几乎完全相同，都需要计算一个 vecdim 维的 SIMD 内积，因此静态划分可以带来最小的调度开销；随后的 nprobe 合并以及 LUT 构建使用了 `#pragma omp single`，single 表示只有一个线程执行这段代码，其他线程等待；最后的 ADC 查表累加使用了 `schedule(dynamic, adc_chunk_size)`，这样更加灵活，根据实际表现调节 `adc_chunk_size` 可以更好的平衡负载开销。

至于 query 部分的 OpenMP 实现也采用了 `schedule(dynamic, query_chunk_size)` 的形式，以求 query 级并行的稳定加速。

7.3.2 实验结果呈现及分析

下面我们通过实验来验证使用 OpenMp 实现的 cluster 并行以及 centroid + cluster 并行的优化效果，实验结果如图 7.9 所示。

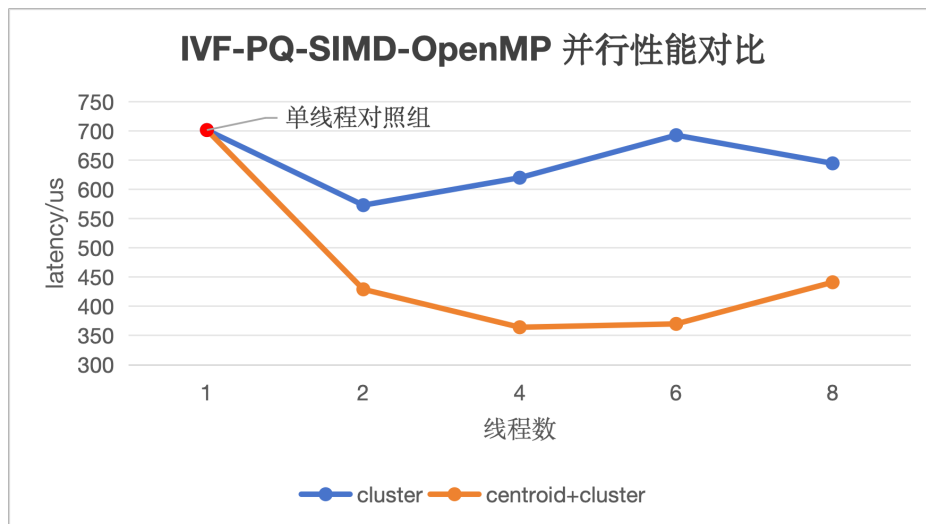


图 7.9: IVF-PQ-SIMD-OpenMP 的多线程并行性能对比 (recall=0.991751)

从实验结果可以看出，单独对 cluster 扫描阶段进行 OpenMP 并行时，整体优化效果并不稳定，这说明在 IVF-PQ 中，仅对 ADC 扫描阶段进行 cluster 级并行时，并行粒度相对有限，同时 PQ 的 ADC 距离计算本身已经较轻量，系统瓶颈容易从计算转向访存、cache 竞争、动态调度和局部堆维护开销。因此，线程数增加后并不一定能够带来更高收益，甚至可能出现负优化现象。

相比之下, centroid+cluster 联合并行的效果明显更好, 其对比 Pthread(见图 7.8) 效果也更突出。这是因为此处的 OpenMP 把两个并行阶段放在同一个 OpenMP parallel region 中执行, 减少了多次线程创建和同步带来的 fork-join 开销, 更能发挥多线程的优势, 实验中, 4 线程 latency 降低到 364.302 us, 相对单线程取得了接近 2 倍的加速。但继续增加线程数后受到 memory-bound、调度开销和串行合并阶段的限制, 性能收益开始下降, 但总体是一个出色的优化。

表 13: IVF-PQ-SIMD-OpenMP query 级并行性能

threads	latency / us	recall	加速比 (对比单线程)
2	356.409	0.991751	1.97
4	181.264	0.991751	3.87
6	125.076	0.991751	5.61
8	92.948	0.991751	7.55

query 级并行优化一如既往的稳定且表现出色, 具体原因这里不再赘述。

经过我们优化之后的 OpenMP 多线程不仅在大计算量的极高 recall 下表现出优势, 在较高 recall(≈ 0.95) 情况下也比 IVF 效果更好, 对比如表 14 所示。

表 14: IVF-PQ-OpenMP 和 IVF 性能对比 (逐 query 搜索, 均采用各自方法最优结果)

方法	threads	latency / us	recall	加速比 (对比 IVF-SIMD)
IVF-SIMD	1	254.077	0.950302	-
IVF-SIMD-Pthread	4	241.953	0.950302	1.05
IVF-PQ-SIMD-OpenMP	6	194.456	0.946153	1.31

8 HNSW

8.1 程序实现思路

HNSW 的核心思想是构建一个分层近邻图。第 0 层包含所有数据点, 图连接较密; 高层只保留部分节点, 用于快速跳转。查询时, 标准 HNSW 会先从最高层入口点出发, 通过贪心搜索快速靠近查询向量所在区域, 然后逐层下降, 最终在第 0 层进行更充分的候选扩展, 得到近似 top-k 结果。

实验中给我们提供了 hnsplib 库, 我们通过调用库函数 searchKNN 实现标准分层查询:

Listing 13: HNSW 标准分层查询

```

1 static inline std::priority_queue<std::pair<float, uint32_t>>
2 hnsw_search_hierarchical(const HNSWIndex& index, const float* query, size_t k, size_t
   ef = 64) {
3     ... ..
4     index.graph->setEf(std::max(k, ef));
5     auto raw = index.graph->searchKnn(query, k);
6     return hnsw_convert_label_result(raw, k);
7 }

```

但据实验手册描述, 我们本次重点探索底层图搜索算法 (Layer0) 及其并行实现, 于是我们先实现了串行搜索的 hnsw_search_layer0, 它不走 HNSW 的上层图, 而是直接从 Layer 0 的某个入口点开始搜索, 对于串行程序, 我们只设置一个入口。

Listing 14: HNSW Layer0 查询

```

1 static inline std::priority_queue<std::pair<float, uint32_t>>
2 hnsw_search_layer0(const HNSWIndex& index, const float* query, size_t k, size_t ef =
   64, int entry_id = -1) {
3     ... ..
4     // 如果没有指定入口就设置一个随机 entry
5     if (entry_id >= 0) {
6         entry = static_cast<uint32_t>(
7             static_cast<size_t>(entry_id) %
8             index.graph->getCurrentElementCount()
9         );
10    } else {
11        entry = hnsw_pick_entry_by_query(index, query);
12    }
13    // 调用底层图搜索库函数
14    auto raw =
        index.graph->searchBaseLayerST<true>(static_cast<hnswlib::tableint>(entry),
        query, real_ef);
15    return hnsw_convert_internal_result(index, raw, k);
16 }

```

实验的重点在于我们使用 Pthread 和 OpenMP 进行了 Layer0 多入口并行优化，多入口点并行即使用多个线程，从不同的入口点同时发起多次完整搜索，最终在主线程中对各线程搜到的 Top-K 结果进行合并排序。该方法可以大幅增加搜索的覆盖面，在很小的 ef 下就能取得极高的 recall，因此在需求高 recall 的情况下性能极佳。

Listing 15: HNSW Layer0 多入口并行查询

```

1 static inline std::priority_queue<std::pair<float, uint32_t>>
2 hnsw_search_layer0_multi_entry_openmp( ... .. ) {
3     ... ..
4     #pragma omp parallel for num_threads(thread_num) schedule(static)
5     for (int i = 0; i < entry_count; ++i) {
6         // 为每个线程选取入口
7         const uint32_t entry =
8             hnsw_pick_entry_by_query(index, query, static_cast<size_t>(i));
9         // 调用底层图搜索库函数
10        auto raw =
            index.graph->searchBaseLayerST<true>(static_cast<hnswlib::tableint>(entry),
            query, real_ef);
11        local_results[static_cast<size_t>(i)] =
12            hnsw_convert_internal_result(index, raw, k);
13    }
14    std::priority_queue<std::pair<float, uint32_t>> result;
15    for (int i = 0; i < entry_count; ++i) {
16        hnsw_merge_topk(result, local_results[static_cast<size_t>(i)], k);
17    }
18    return result;
19 }

```


8.2 实验结果呈现及分析

我们通过实验先来测试 hnsplib 中已经实现的标准分层查询函数和 Layer0 查询函数的性能，结果如图 8.10 所示。

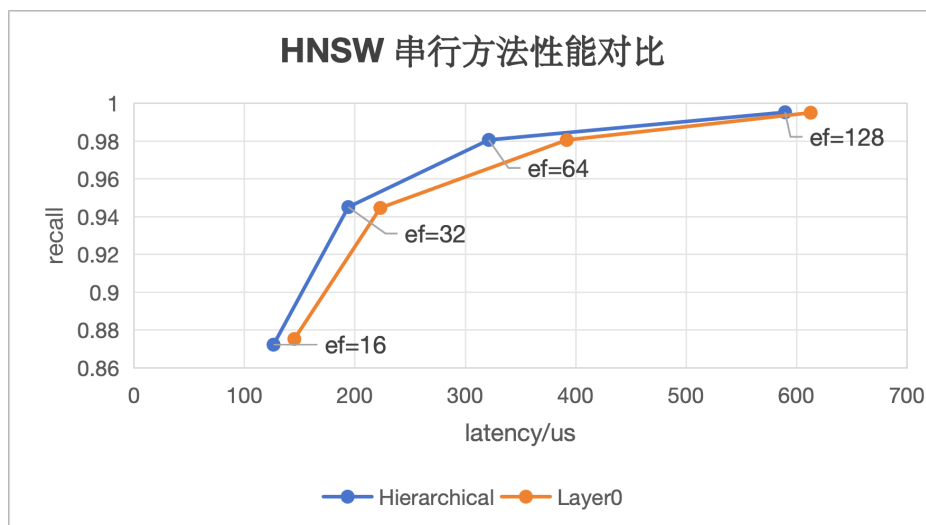


图 8.10: HNSW 串行方法性能对比

在 HNSW 查询过程中, ef 参数用于控制搜索阶段候选集合的规模, 是需要调节的核心参数。较小的 ef 会使算法只扩展少数最近节点, latency 较低, 但容易陷入局部最优; 随着 ef 增大, 算法能够探索更多的搜索路径, 从而提高 recall, 但计算量也会增加, latency 会升高。借助串行方法, 我们也通过调整合适的 ef 来进行 recall 和 latency 的 trade-off。从 HNSW 串行方法性能对比可以看出, 标准 Hierarchical 搜索在几乎相同 recall 下通常具有更低 latency, 说明 HNSW 的高层图能够发挥有效的导航作用, 帮助搜索更快到达近邻区域, 但总体提升也很有限。Layer0 方法虽然跳过了高层搜索, 但由于入口点质量不稳定, 需要在底层图中进行更多扩展, 因此整体性能略差。

下面我们通过实验重点来看采用了多入口并行的 HNSW 算法的性能表现, 结果如图 8.11 所示。

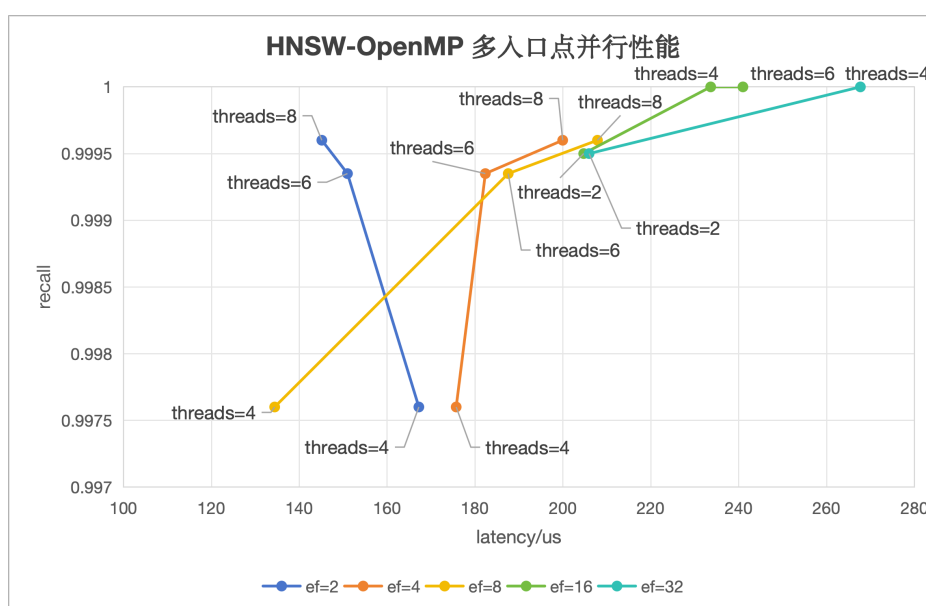


图 8.11: HNSW-OpenMP 多入口点并行

从 OpenMP 多入口点并行实验可以看出,多线程参与搜索让 recall 明显提高,都在低延迟下达到了 0.995 以上的水平,说明多个入口点能够扩大 Layer0 搜索覆盖范围,降低单入口搜索陷入局部区域的风险。然而 latency 并没有随线程数增加而单调下降,部分情况下甚至明显上升。这是因为该并行方法并不是对固定搜索任务进行简单划分,而是让多个线程从不同入口点执行独立搜索,线程数增加的同时也增加了总搜索量。此外,HNSW 搜索具有随机访存、分支判断和候选队列维护等特征,容易受到 cache miss、内存带宽和线程调度开销影响,因此并行加速并不稳定。总体来看,多入口 OpenMP 版本能够以较低 latency 获得极高 recall,效果出色。

在实现 OpenMP 的同时,我们也实现了 Pthread 方法的多入口并行,下面通过一张表格来汇总我们 HNSW 的表现优异的实验结果,见表 15。

表 15: HNSW 不同方法性能对比

方法	threads	latency / us	recall
标准分层查询	1	194.016	0.945154
底层图搜索查询	1	223.043	0.944704
多入口并行查询-Pthread	8	441.286	0.9996
多入口并行查询-OpenMP	8	145.123	0.9996

9 实验总结与思考

本次实验围绕 ANNS 向量检索任务,在已有 SIMD 优化的基础上,进一步使用 Pthread 和 OpenMP 对 Flat、PQ、IVF、IVF-PQ 和 HNSW 等方法进行了多线程并行优化,并通过 latency 和 recall 分析不同优化策略的效果,不同的方法都有不同的优劣和适合的应用场景。

从多线程优化方式来看,Pthread 和 OpenMP 二者特点不同。Pthread 需要手动完成线程创建、任务划分、同步等待和结果合并,因此控制能力更强,适合实现自定义线程池、动态任务队列等复杂逻辑。例如在 PQ 的 LUT 构建和 IVF 的动态簇划分中,可以根据任务特点手动设计线程参数和同步方式。但 Pthread 编程复杂度较高,如果每个 query 都频繁创建和销毁线程,会带来明显额外开销。相比之下,OpenMP 通过简单的并行指令即可完成循环并行、线程调度和同步,代码实现更加简洁,调度机制更加成熟,往往能取得更稳定的加速效果。

从实验结果来看,不同 ANNS 方法的并行效果与其计算特征密切相关,多线程加速也不意味着随着线程数增多就一定取得更好的效果。对于计算量大,计算密集的任务,如 Flat-SIMD,就比 PQ-SIMD 多线程加速效果更好。多线程加速也取决于具体参数情况,例如 IVF-SIMD 通过倒排索引减少候选数量,簇并行在 nprobe 较大时更有效,而 query 级并行由于任务独立,表现最稳定;IVF-PQ 在候选集较大时相比 IVF 更有优势,不然反而会造成负优化。

通过本次实验,我更加深入地理解了并行优化不能只看线程数,而要结合算法瓶颈、任务粒度、负载均衡和访存模式综合分析。同时我也能更加理性的看待实验结果,不会因为负优化而产生疑虑,而是会静下心来细细分析当中蕴藏的原因。